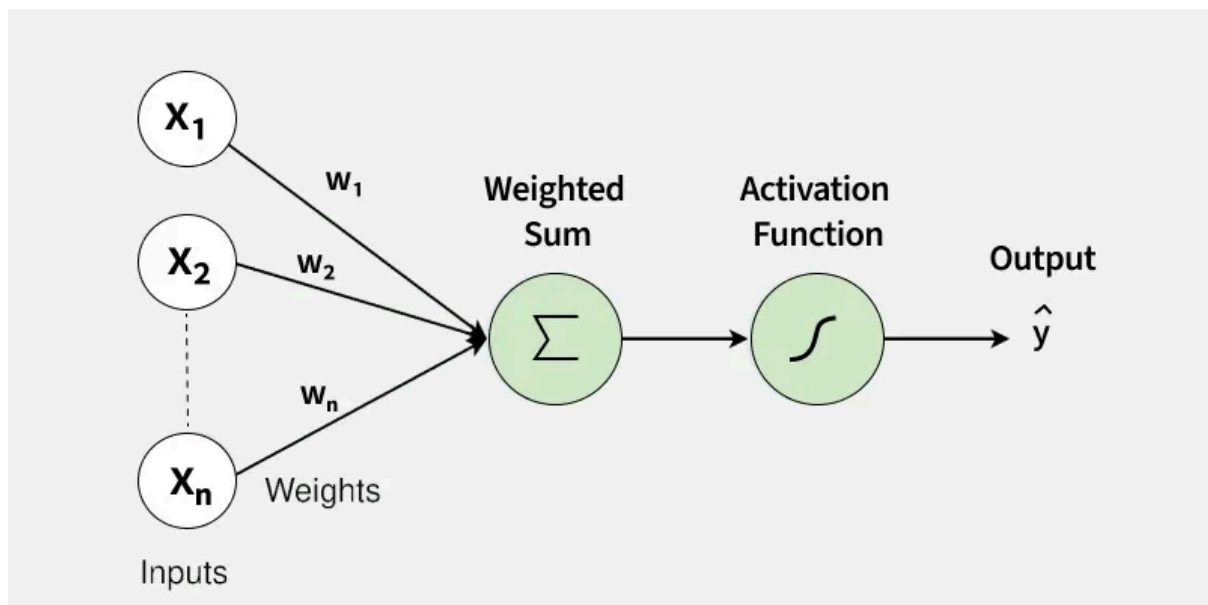


What is Perceptron

A Perceptron is the simplest form of a neural network that makes decisions by combining inputs with weights and applying an activation function. It is mainly used for binary classification problems. It forms the basic building block of many deep learning models.

- Takes multiple inputs and assigns weights
- Computes a weighted sum and applies a threshold
- Outputs either 0 or 1 (binary outcome)
- Forms the foundation of larger neural networks

Core Components



Perceptron

1. Inputs (x_n)

These are the features or measurable attributes of a data point that the perceptron uses to make a decision. Each input provides a signal that contributes to the final output.

- **Example:** For an OR gate, the inputs are binary: $(x_1, x_2) \in \{0, 1\}^2$
- Inputs themselves have no inherent influence unless multiplied by weights.

2. Weights (w_n)

Weights determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.

- Weights are learned during training, adjusting based on errors.

- They act like importance scores for each feature.

3. Bias (b)

The bias is a constant value added to the weighted sum to shift the decision boundary.

- It allows the perceptron to classify correctly even when all input features are zero.
- Bias ensures the model is not forced to pass the decision boundary through the origin.

Difference Between Weights and Bias

- Weights control how much each input influences the output.
- Bias controls when the perceptron activates, independent of any input.
- Mathematically, Weights tilt the line and Bias shifts the line up/down or left/right.

4. Net Input (Weighted Sum)

This is the combined effect of all inputs and their weights:

$$z = \sum_{i=1}^n w_i x_i + b$$

- Represents the activation strength before passing through the activation function.
- If z is high or low enough, it determines the final class.

5. Activation Function (Step Function)

The activation function converts the numerical input into a binary output:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- It introduces non-linearity in the decision-making, although the decision boundary remains linear.
- Output is always 0 or 1 making perceptron's suitable for binary classification.

Learning Laws:

1. Perceptron Model (Recall)

$$y = f(b)$$

Where:

- $x_i \rightarrow$ inputs
- $w_i \rightarrow$ weights
- $b \rightarrow$ bias

- $f(\cdot) \rightarrow$ activation function (step function)

2. Perceptron Learning Law (Error-Correction Rule)

This is the primary learning law used in perceptrons.

Rule

$$\Delta w_i = \eta(t - y)x_i \Delta b = \eta(t - y)$$

Where:

- $\eta \rightarrow$ learning rate
- $t \rightarrow$ target output
- $y \rightarrow$ actual output

Weight Update

$$w_i^{new} = w_i^{old} + \Delta w_i$$

Working Principle

- If $t = y \rightarrow$ no change in weights
- If $t \neq y \rightarrow$ weights are adjusted to reduce error

3. Hebbian Learning Law

A biologically inspired learning law often discussed with perceptrons.

Rule

$$\Delta w_i = \eta x_i y$$

Characteristics

- Unsupervised learning
- No error correction
- Strengthens frequently used connections

4. Delta Learning Law (Widrow–Hoff Rule)

Used when output is continuous instead of binary.

Error Function

$$E = \frac{1}{2} (t - y)^2$$

Rule

$$\Delta w_i = \eta(t - y)x_i$$

- ✓ Minimizes mean squared error (MSE)
 - ✓ Basis for gradient descent
-

5. Comparison of Learning Laws

Learning Law	Type	Error-based	Used in
Hebbian	Unsupervised	✗	Association learning
Perceptron	Supervised	✓	Binary classification
Delta rule	Supervised	✓	Continuous output

6. Convergence Property

The Perceptron Convergence Theorem states that the perceptron learning algorithm converges in a finite number of steps if and only if the training data is linearly separable.

7. Limitations of Perceptron Learning

- Cannot learn non-linearly separable problems (e.g., XOR)
- Uses only single-layer architecture
- Binary output only

1. Hebbian Learning Law (Unsupervised)

Rule

$$\Delta w = \eta xy$$

Given

- Input $x = 1$
- Output $y = 1$
- Learning rate $\eta = 0.1$
- Initial weight $w = 0.5$

Calculation

$$\Delta w = 0.1 \times 1 \times 1 = 0.1 w^{new} = 0.5 + 0.1 = 0.6$$

Key Point

- ✓ Weight increases when input and output are active together
 - ✓ No target output required
-

2. Perceptron Learning Law (Supervised, Binary Output)

Rule

$$\Delta w = \eta(t - y)x$$

Given

- Input $x = 1$
- Target $t = 1$
- Actual output $y = 0$
- Learning rate $\eta = 0.2$
- Initial weight $w = 0.3$

Calculation

$$\Delta w = 0.2(1 - 0)(1) = 0.2 w^{new} = 0.3 + 0.2 = 0.5$$

Key Point

- ✓ Weight updated only when output is wrong
-

3. Delta Learning Law (Widrow–Hoff Rule)

Rule

$$\Delta w = \eta(t - y)x$$

Given

- Input $x = 1$
- Target $t = 1$
- Actual output $y = 0.6$
- Learning rate $\eta = 0.1$
- Initial weight $w = 0.4$

Calculation

$$\Delta w = 0.1(1 - 0.6)(1) = 0.04w^{new} = 0.4 + 0.04 = 0.44$$

Key Point

- ✓ Works with continuous output
- ✓ Minimizes mean squared error

Quick Comparison (One-Glance Revision ★)

Learning Law	Uses Target?	Output Type	Weight Change
Hebbian	✗ No	Any	Based on x, y
Perceptron	✓ Yes	Binary	Based on error
Delta	✓ Yes	Continuous	Gradient-based



Fundamentals of Neural Network

A neural network extends the perceptron by connecting many neurons across multiple layers.

1. Input layer: The input layer provides the network with the raw feature vector:

$$x = (x_1, x_2, \dots, x_n)$$

- No computation happens here.
- It simply passes the input values to the next layer.

Input Layer work

- Receives raw data
- No computation
- One neuron per feature

2. Hidden layers: Hidden layers contain multiple perceptron's (neurons) that learn intermediate representations of the data.

Hidden Layer Computation:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(1)} = \sigma(z^{(1)})$$

where:

- $W^{(1)}$: weight matrix for hidden layer
- $b^{(1)}$: bias vector
- σ : non-linear activation function (ReLU, Sigmoid, Tanh, etc.)
- Hidden layers identify complex patterns not visible from raw input alone.
- Adding more hidden layers improves model expressiveness.

Hidden Layer(s)

- Perform feature extraction
- Introduce non-linearity
- Can be **one or more**

Activation functions commonly used:

- Sigmoid

- Tanh
- ReLU

3. Output layer: The output layer produces the final prediction, which may be binary, multi-class or a continuous value.

Output Layer Computation:

$$z^{(2)} = W^{(2)} a^{(1)} + b^{(2)}$$

$$\hat{y} = \sigma(z^{(2)})$$

Output activation depends on the task:

- **Sigmoid:** binary classification
- **SoftMax:** multi-class classification
- **Linear:** regression

Because of multiple layers and non-linear activations, neural networks can model complex, non-linear decision boundaries, while a single perceptron can only model a straight line.

3.3 Output Layer

- **Produces final prediction**
- **Activation depends on task:**
 - **Binary classification** → **Sigmoid**
 - **Multi-class** → **Softmax**
 - **Regression** → **Linear**

Working

Training a perceptron means finding suitable weights w_i and bias b such that most training points are correctly classified.

1. Compute the Weighted Sum

The perceptron first calculates a weighted combination of the input features, along with a bias term that helps shift the decision boundary.

$$z = \sum_{i=1}^n w_i x_i + b$$

2. Apply the Activation Function (Step Function)

The perceptron uses a simple threshold activation to convert the numerical value into a binary class label.

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

3. Compare Prediction with Actual Output

The perceptron checks if the predicted output matches the true label.

$$error = y - \hat{y}$$

4. Update the Weights (Learning Rule)

Whenever the perceptron misclassifies a sample, it updates each weight by an amount proportional to the error and the input value.

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

5. Update the Bias Term

The bias is adjusted similarly to shift the decision boundary left or right.

$$b \leftarrow b + \eta(y - \hat{y})$$

6. Repeat for All Samples Across Multiple Epochs

The perceptron cycles through the entire dataset several times (epochs), refining weights gradually until it reaches a stable solution.

7. Final Learned Model

After training, the perceptron produces predictions using:

$$\hat{y} = \text{step}(w^\top x + b)$$

Types of ANN Based on Layers

Network Type	Description
Single-layer	Only input and output
Multi-layer	One or more hidden layers
Deep Neural Network	Many hidden layers

Aspect	Perceptron	Multi-Layer Perceptron (MLP)
Model Depth	Single layer with no hidden neurons.	Multiple layers with one or more hidden layers.
Type of Patterns Learned	Learns only linear relationships; straight-line separation.	Learns complex, non-linear patterns and curved boundaries.
Problem-Solving Ability	Cannot solve XOR or non-linearly separable problems.	Easily solves XOR and other complex classification tasks.
Activation Functions	Uses a simple step function (hard 0/1 output).	Uses advanced activations like ReLU, Sigmoid, Tanh for richer learning.
Learning Method	Trained with a simple perceptron update rule.	Trained using backpropagation and gradient descent.
Real-World Use	Limited to simple demonstrations and basic classification.	Used in real-world AI systems like vision, NLP and deep learning applications.

Applications

- **Binary Classification:** Used for simple yes/no decision problems such as spam detection or basic quality checks.
- **Logic Gate Modelling:** Implements linearly separable gates like AND or NAND with perfect accuracy.
- **Pattern Recognition Basics:** Helps identify simple patterns where classes can be separated with a straight line.

- **Feature Importance Insight:** Weight values indicate which features influence the output more strongly.
- **Educational Tool:** Commonly used to teach foundations of machine learning and neural networks.

Advantages

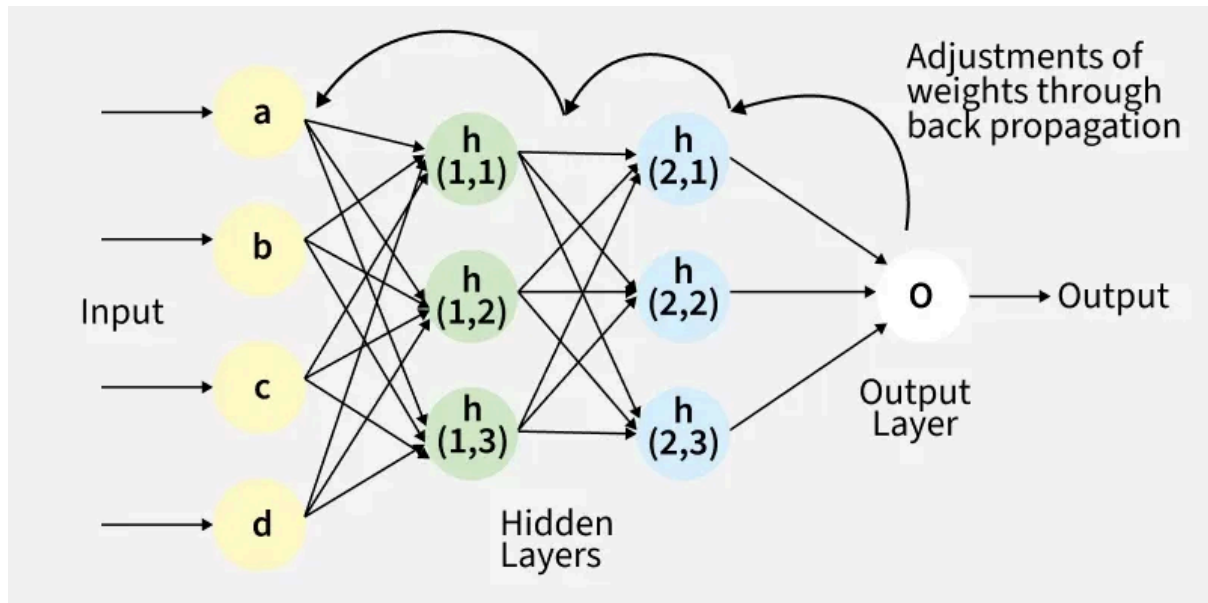
- **Simple Architecture:** Easy to understand, implement and visualize as a basic neural model.
- **Fast Training:** Lightweight computations make learning very quick even on small devices.
- **Works on Linearly Separable Data:** Achieves perfect performance when a straight-line boundary exists.
- **Low Resource Requirement:** Requires minimal memory and computation, suitable for tiny datasets.
- **Foundation for Deep Learning:** Forms the conceptual basis for multilayer perceptron's and complex networks.

Limitations

- **No Non-Linear Learning:** Fails on problems like XOR where a curved or complex boundary is needed.
- **Binary Output Only:** Cannot handle multi-class or probabilistic outcomes without extensions.
- **No Hidden Layers:** Lacks depth, making it unable to learn hierarchical or abstract patterns.
- **Sensitive to Data Scale:** Performance drops if features are not normalized or scaled properly.
- **Not Suitable for Real-World ML:** Too simplistic for modern tasks like vision, NLP or sequence modeling.

Backpropagation in Neural Network

Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs. It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases. Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.



Fig(a) A simple illustration of how the backpropagation works by adjustments of weights

Back Propagation plays a critical role in how neural networks improve over time. Here's why:

1. **Efficient Weight Update:** It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
2. **Scalability:** The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
3. **Automated Learning:** With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

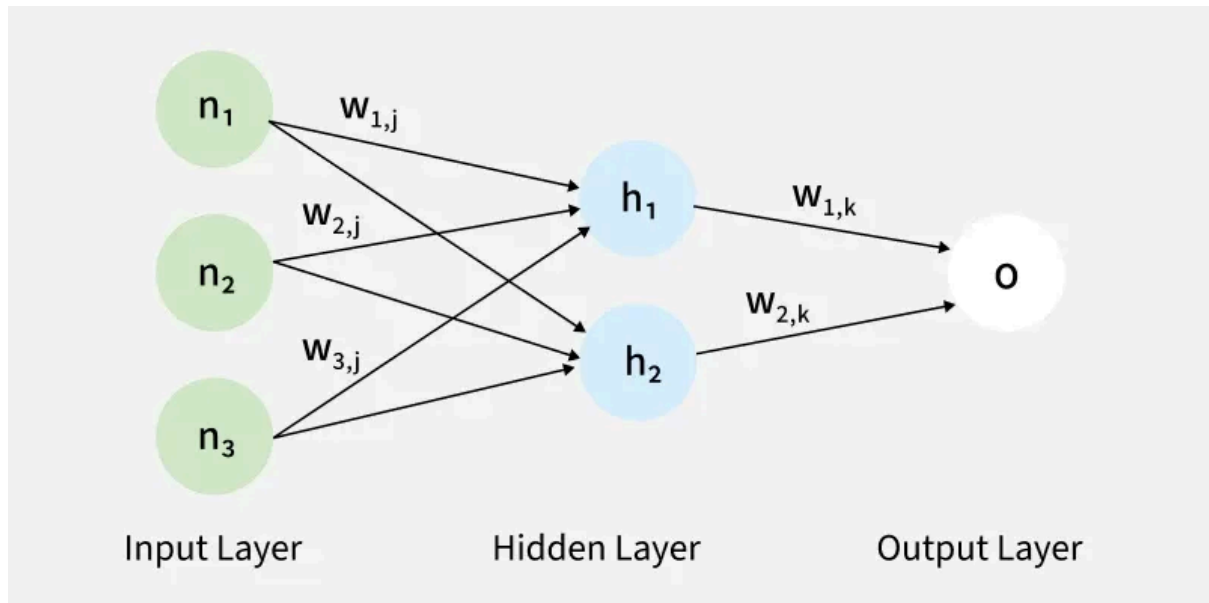
Working of Back Propagation Algorithm

The Back Propagation algorithm involves two main steps: the Forward Pass and the Backward Pass.

1. Forward Pass Work

In forward pass the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2. Before applying an activation function, a bias is added to the weighted inputs.

Each hidden layer computes the weighted sum ('a') of the inputs then applies an activation function like [ReLU \(Rectified Linear Unit\)](#) to obtain the output ('o'). The output is passed to the next layer where an activation function such as [softmax](#) converts the weighted outputs into probabilities for classification.



The forward pass using weights and biases

2. Backward Pass

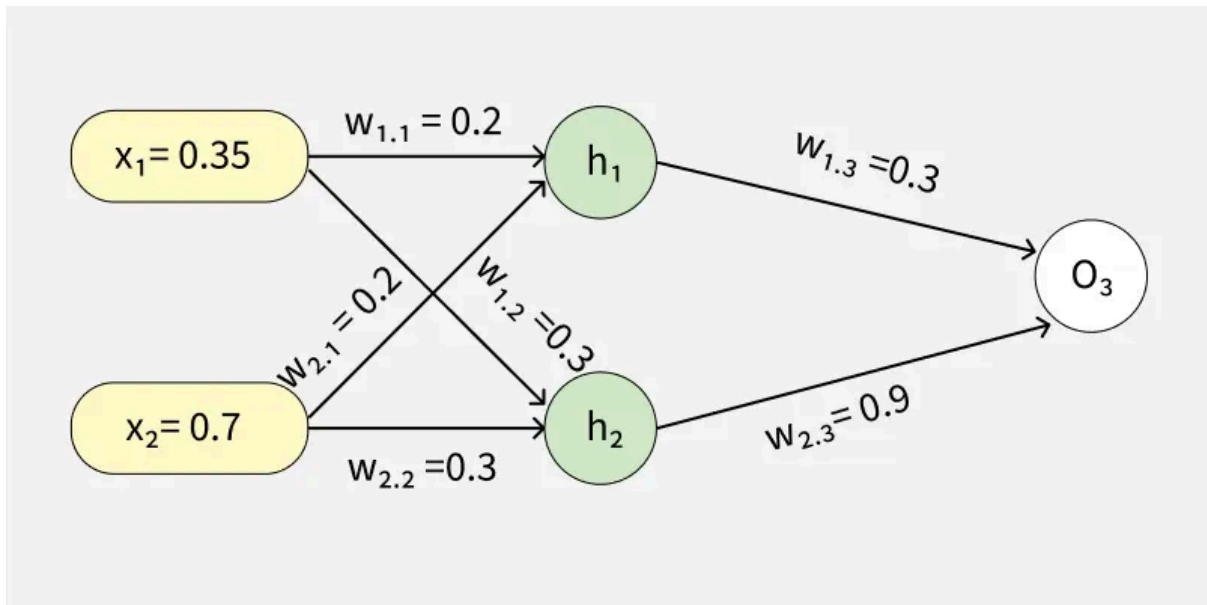
In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the [Mean Squared Error \(MSE\)](#) given by:

$$MSE = (\text{Predicted Output} - \text{Actual Output})^2$$

Once the error is calculated the network adjusts weights using gradients which are computed with the chain rule. These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration. The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.

Example of Back Propagation in Machine Learning

Let's walk through an example of Back Propagation in machine learning. Assume the neurons use the sigmoid activation function for the forward and backward pass. The target output is 0.5 and the learning rate is 1.



Example (1) of backpropagation sum

Forward Propagation

1. Initial Calculation

The weighted sum at each node is calculated using:

$$a_j = \sum(w_{i,j} * x_i)$$

Where,

- a_j is the weighted sum of all the inputs and weights at each node
- $w_{i,j}$ represents the weights between the i^{th} input and the j^{th} neuron
- x_i represents the value of the i^{th} input

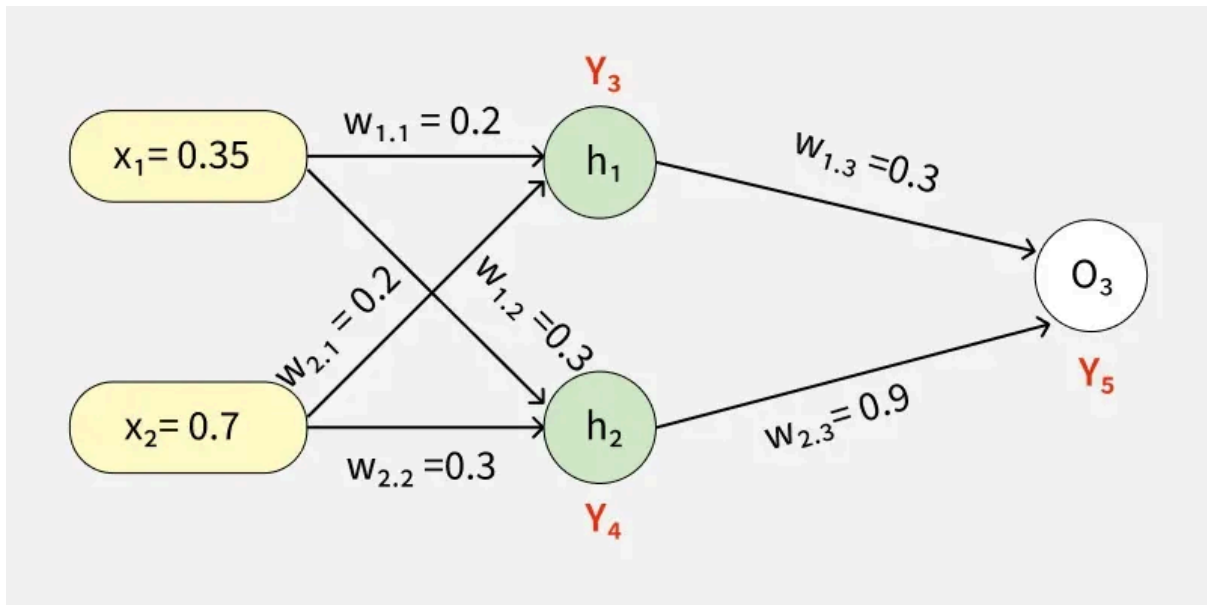
O (output): After applying the activation function to a, we get the output of the neuron:

$$o_j = \text{activation function}(a_j)$$

2. Sigmoid Function

The sigmoid function returns a value between 0 and 1, introducing non-linearity into the model.

$$y_j = \frac{1}{1+e^{-a_j}}$$



To find the outputs of y_3 , y_4 and y_5

3. Computing Outputs

At h_1 node

$$a_1 = (w_{1,1}x_1) + (w_{2,1}x_2) = (0.2 * 0.35) + (0.2 * 0.7) = 0.21$$

Once we calculated the a_1 value, we can now proceed to find the y_3 value:

$$y_j = F(a_j) = \frac{1}{1+e^{-a_j}}$$

$$y_3 = F(0.21) = \frac{1}{1+e^{-0.21}}$$

$$y_3 = 0.56$$

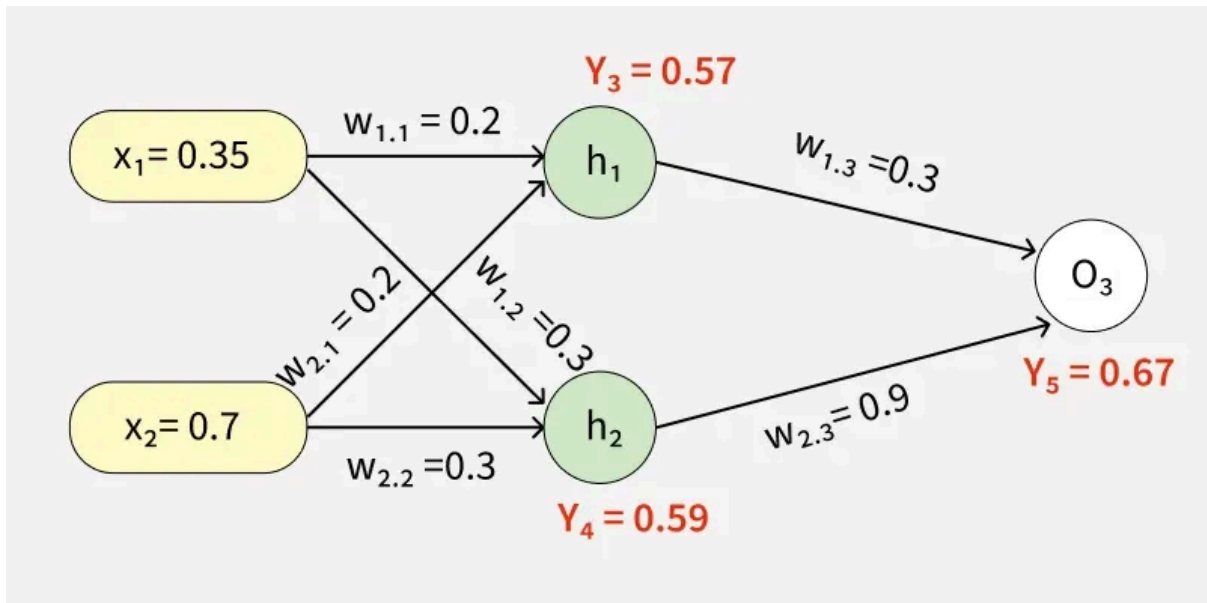
Similarly find the values of y_4 at h_2 and y_5 at O_3

$$a_2 = (w_{1,2} * x_1) + (w_{2,2} * x_2) = (0.3 * 0.35) + (0.3 * 0.7) = 0.315$$

$$y_4 = F(0.315) = \frac{1}{1+e^{-0.315}} = 0.578$$

$$a_3 = (w_{1,3} * y_3) + (w_{2,3} * y_4) = (0.3 * 0.57) + (0.9 * 0.59) = 0.702$$

$$y_5 = F(0.702) = \frac{1}{1+e^{-0.702}} = 0.67$$



Values of y_3 , y_4 and y_5

4. Error Calculation

Our actual output is 0.5 but we obtained 0.67. To calculate the error we can use the below formula:

$$\text{Error}_j = y_{\text{target}} - y_5$$

$$\Rightarrow 0.5 - 0.67 = -0.17$$

Using this error value we will be backpropagating.

Back Propagation

1. Calculating Gradients

The change in each weight is calculated as:

$$\Delta w_{ij} = \eta \times \delta_j \times O_j$$

Where:

- δ_j is the error term for each unit,
- η is the learning rate.

2. Output Unit Error

For O_3 :

$$\delta_5 = y_5(1 - y_5)(y_{\text{target}} - y_5)$$

$$= 0.67(1 - 0.67)(-0.17) = -0.0376$$

3. Hidden Unit Error

For h1:

$$\begin{aligned}\delta_3 &= y_3(1 - y_3)(w_{1,3} \times \delta_5) \\ &= 0.56(1 - 0.56)(0.3 \times -0.0376) = -0.0027\end{aligned}$$

For h2:

$$\begin{aligned}\delta_4 &= y_4(1 - y_4)(w_{2,3} \times \delta_5) \\ &= 0.59(1 - 0.59)(0.9 \times -0.0376) = -0.0819\end{aligned}$$

4. Weight Updates

For the weights from hidden to output layer:

$$\Delta w_{2,3} = 1 \times (-0.0376) \times 0.59 = -0.022184$$

New weight:

$$w_{2,3}(\text{new}) = -0.022184 + 0.9 = 0.877816$$

For weights from input to hidden layer:

$$\Delta w_{1,1} = 1 \times (-0.0027) \times 0.35 = 0.000945$$

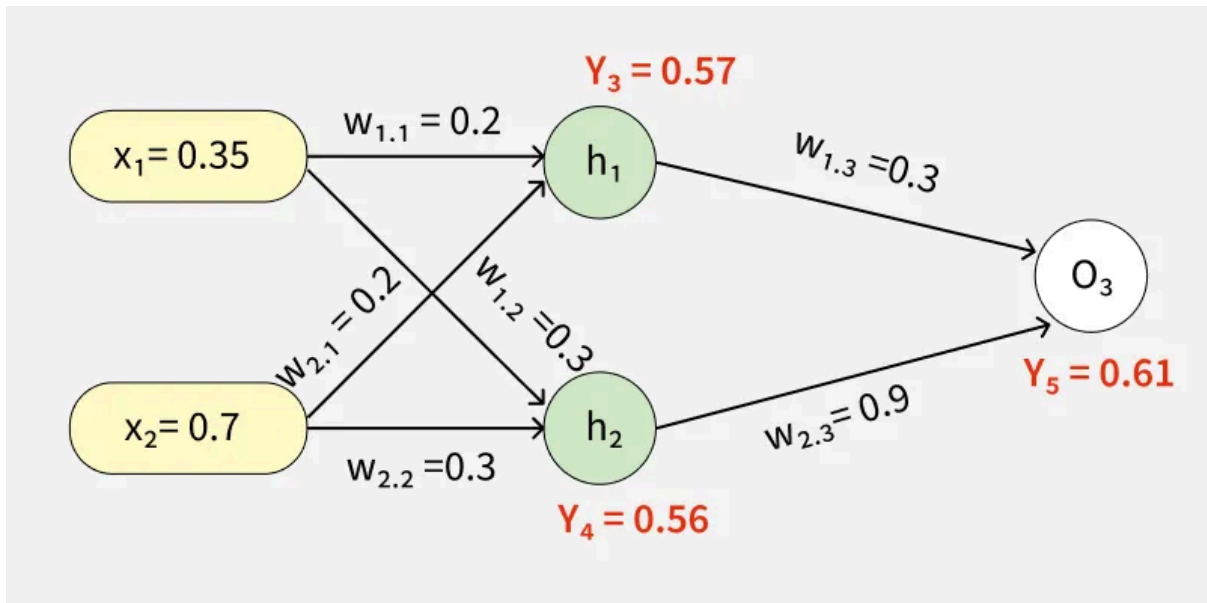
New weight:

$$w_{1,1}(\text{new}) = 0.000945 + 0.2 = 0.200945$$

Similarly other weights are updated:

- $w_{1,2}(\text{new}) = 0.273225$
- $w_{1,3}(\text{new}) = 0.086615$
- $w_{2,1}(\text{new}) = 0.269445$
- $w_{2,2}(\text{new}) = 0.18534$

The updated weights are illustrated below



Through backward pass the weights are updated

After updating the weights the forward pass is repeated hence giving:

- $y_3 = 0.57$
- $y_4 = 0.56$
- $y_5 = 0.61$

Since $y_5 = 0.61$ is still not the target output the process of calculating the error and backpropagating continues until the desired output is reached.

This process demonstrates how Back Propagation iteratively updates weights by minimizing errors until the network accurately predicts the output.

$$\begin{aligned} \text{Error} &= y_{\text{target}} - y_5 \\ &= 0.5 - 0.61 = -0.11 \end{aligned}$$

This process is said to be continued until the actual output is gained by the neural network.